

Երկակի գործակալի դիլեման

- Ժամանակի սահմանափակում. 12 րոպե:
- Հիշողություն. 5 ԳԲ
- Միջավայր. մեկ GPU (≈ 16 ԳԲ VRAM), առանց ինտերնետի
- Լուծման չափը. `solution.ipynb` ≤ 1 ՄԲ
- **Baseline score.** 0

Աստանայի AI ազգային կենտրոնում երկու համակարգչային մոդելներ՝ Model R-ը (ResNet-18) և Model V-ն (ViT-Tiny), վերլուծում են լուսանկարներ: Այս պահին երկու մոդելներն էլ կատարյալ աշխատանք են անում՝ ապահովելով 100% accuracy (ճշգրտություն) և համաձայնելով յուրաքանչյուր նկարի հարցում: Ստուգելու համար, թե որքան տարբեր են իրականում նրանց խելացի «ուղեղները», գլխավոր գիտնականը ձեռք մարտահրավեր է նետում. յուրաքանչյուր լուսանկարում կատարել փոքրիկ, գրեթե աննկատելի փոփոխություններ, որպեսզի Model R-ը և Model V-ն լիովին չհամաձայնեն միմյանց հետ:



1. Առաջադրանք

Նախապես վերապատրաստված պատկերների երկու դասակարգիչներ (pretrained image classifiers) նայում են միևնույն պատկերին: Այս առաջադրանքում տրամադրված

պատկերների վրա երկու դասակարգիչներն էլ աշխատում են 100% accuracy-ով:

- **Model R.** `torchvision.models.resnet18` (CNN, ResNet18):
- **Model V.** `timm-ի vit_tiny_patch16_224` (Transformer, ViT-Tiny):

Ձեր առաջադրանքն է յուրաքանչյուր պատկերի համար ստեղծել փոքր փոփոխություն («խտտորում» / perturbation), որպեսզի երկու մոդելները չհամաձայնեն միմյանց հետ: Յուրաքանչյուր պատկերի համար դուք պետք է ստեղծեք **երկու տարբեր** perturbation-ներ.

- **Type A.** այն ավելացնելուց հետո Model R-ը դեռևս ճիշտ է դասակարգում պատկերը, իսկ Model V-ն դասակարգում է սխալ:
- **Type B.** այն ավելացնելուց հետո Model V-ն դեռևս ճիշտ է դասակարգում պատկերը, իսկ Model R-ը դասակարգում է սխալ:

Յուրաքանչյուր perturbation պետք է լինի բավականաչափ *փոքր*, որպեսզի դժվար լինի նկատել: Ավելի փոքր perturbation-ները ստանում են ավելի բարձր միավորներ (տես Բաժին 5-ը): Perturbation-ը կիրառվում է բնօրինակ պատկերի վրա անմիջապես փիքսելների մակարդակով:

2. Հանրային տվյալներ (Public data)

Առաջադրանքի հետ տրամադրվում է պատկերների բազմություն՝ բաժանված երկու split-ի՝ `train` (100 պատկեր) և `test_public` (100 պատկեր), որոնցից յուրաքանչյուրում կան տարբեր resolution-ով պատկերներ: Բոլոր պատկերները ImageNet-1K-ի 1000 դասերից են, և թե՛ Model R-ը, թե՛ Model V-ն հասնում են 100% accuracy-ի երկու split-ներում էլ:

Տրամադրված են հետևյալ ֆայլերը.

```
train/images/*.png          # 100 images in PNG format
train/labels.json           # maps each image index to its correct class
test_public/images/*.png    # 100 images in PNG format
test_public/labels.json     # maps each image index to its correct class
```

Գնահատման ընթացքում ձեր `dataset/test_public/` թղթապանակը թափանցիկորեն փոխարինվում է պատկերների երկու թաքնված բազմություններով (`test_leaderboard_a` և `test_leaderboard_b`)՝ պաշտոնական գնահատման համար: Դրանցից յուրաքանչյուրը պարունակում է **100 պատկեր** PNG ձևաչափով և label ֆայլ:

Նշում. Այս առաջադրանքի համար **test dataset-ներում label-ները հասանելի են:**

3. Ելքային ձևաչափ (Output format)

Յուրաքանչյուր պատկերի համար դուք պետք է ստեղծեք երկու ֆայլ.

```
{index}_a.pt    # Type A perturbation
{index}_b.pt    # Type B perturbation
```

- {index} (0, 1, 2, ...), համապատասխանում է dataset-ներում պատկերի անվանը:
- Յուրաքանչյուր ֆայլ մեկ tensor է՝ պահպանված `torch.save`-ով: Դրա shape-ը պետք է լինի $3 \times H \times W$, որտեղ H -ը և W -ը համապատասխանում են տվյալ պատկերի **բնօրինակ** resolution-ին (ոչ թե 224×224):
- Կողը պետք է գեներացնի միայն մեկ ZIP ֆայլ՝ `submission.zip`: Տեղադրեք բոլոր `.pt` ֆայլերը ZIP արխիվի վերին մակարդակում (top level), առանց որևէ պարունակող թղթապանակի կամ ենթաթղթապանակների:

```
submission.zip
├─ 0_a.pt
├─ 0_b.pt
├─ 1_a.pt
└─ ...
```

Notebook-ը ձեզ կգուշացնի, եթե ելքային ձևաչափի հետ կապված որևէ խնդիր լինի:

4. Սահմանափակումներ

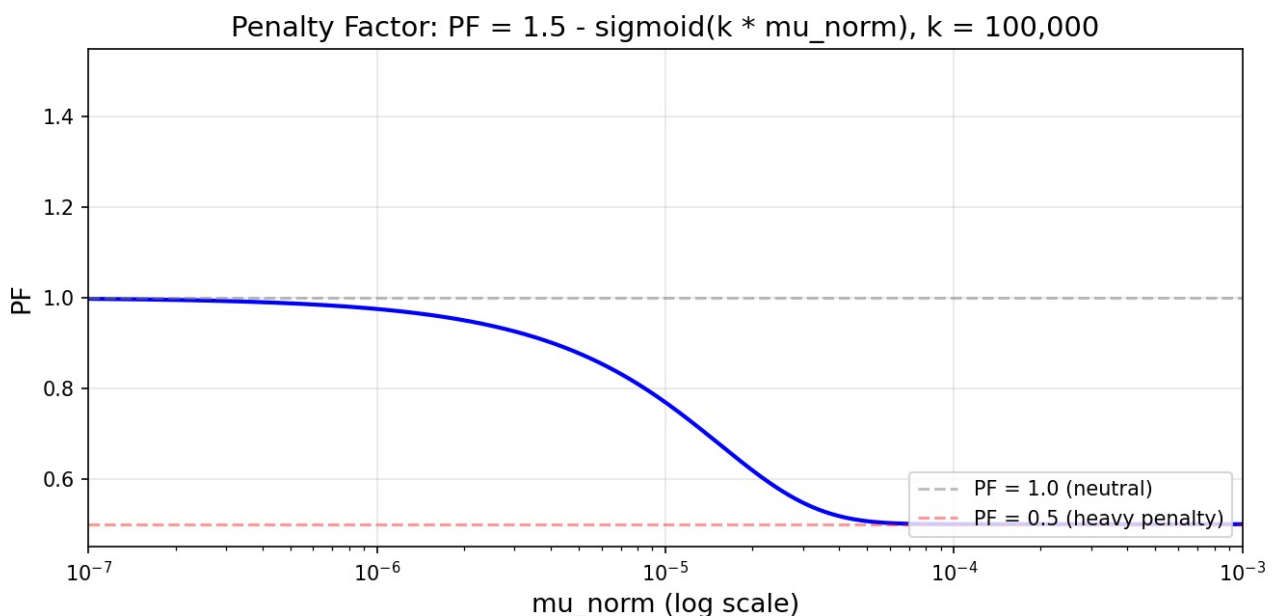
- **Մոդելներ.** Դուք պետք է օգտագործեք `torchvision.models.resnet18(pretrained=True)` և `timm.create_model('vit_tiny_patch16_224', pretrained=True)`: Այլ pretrained մոդելներ չեն թույլատրվում:
- **Transform pipeline** (պարտադրվում է գնահատման ժամանակ). `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])`. Տես Section 3 ում `baseline.ipynb`՝ մանրամասների համար:
- **Perturbation resolution.** Պետք է համապատասխանի բնօրինակ չմշակված (raw) պատկերի resolution-ին (ոչ թե 224×224): Tensor-ն ավելացվում է raw պատկերին transform pipeline-ից առաջ:
- **Ելքային ձևաչափ.** Միայն `.pt` ֆայլեր — առանց PNG/JPG-ի: Tensor-ներն ավելացվում են raw պատկերին, և փիքսելների արժեքները clipped են արվում `[0, 1]` միջակայքում՝ preprocessing-ից առաջ:
- **Ֆայլերի անվանում.** Հարթ ցուցակով (flat-listed), խիստ `{index}_a.pt` / `{index}_b.pt` ձևաչափով: ZIP-ի ներսում ենթաթղթապանակներ չպետք է լինեն:
- **Գրադարաններ.** `torch`, `torchvision`, `timm`:

5. Գնահատում (Scoring)

Վերջնական միավորը հաշվարկվում է հետևյալ կերպ. Դիցուք M -ը split-ում պատկերների քանակն է, $Score_A$ -ը՝ հաջողված Type A perturbation-ների քանակը, իսկ $Score_B$ -ը՝ հաջողված Type B perturbation-ների քանակը.

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PF-ն ֆունկցիա է, որը նախագծված է բարձր նորմ (norm) ունեցող perturbation-ները պատժելու և կատարողականի առավելագույն սահմանի մոտ շատ զգայուն լինելու համար: Այն սահմանափակված է 0.5-ից 1 միջակայքում: Ամբողջական իրականացումը (implementation) կարող եք տեսնել `solution.ipynb`-ի 8-րդ բաժնում:



Նկար. Տուգանային ֆունկցիայի (penalty function) կորը:

6. Ստուգեք ուղարկվող լուծումը (Check the Submission)

Notebook-ում կան ստուգումներ, որոնք զգուշացնում են ձեզ ձևաչափման խնդիրների առկայության մասին՝ `solution.ipynb` notebook-ի 7-րդ բաժնում:

7. Տեղական տեստավորում (Local testing)

`solution.ipynb`-ը պարունակում է ամբողջական, աշխատող օրինակ: Այն բեռնում է public data-ն, երկու մոդելները և պաշտոնական scorer-ը, ապա գրում է submission ZIP ֆայլ: Կարդացեք այն նախքան սկսելը:

8. Ինչպես ուղարկել լուծումը (How to submit)

- Պահպանեք ձեր փոփոխությունները `solution.ipynb`-ում:
- Բացեք Git ներդիրը JupyterLab-ի ձախ կողմնային վահանակում (sidebar):
- **Stage** արեք `solution.ipynb`-ը (դրա կողքի + նշանը):
- Մուտքագրեք `commit` հաղորդագրություն և սեղմեք **Commit**:
- Սեղմեք դեպի վեր սլաքով ամպի նշանը՝ `push` անելու համար:
- Վերադարձեք Contest-ի այս էջը և սեղմեք **Submit**:

Ուղարկեք ճիշտ մեկ ֆայլ՝ `solution.ipynb` անունով: